

Heroku Pipelines for CD - Continuous Delivery

Original Author: Yihao Xie, TA, CSCE 431 - Spring 2021, Fall 2021

Revision: Pauline Wade, CSCE 431 - Spring 2025

Riddhi Ghate, TA, CSCE 431 – Fall 2025

Pipelines are extremely useful for [managing multiple environments for your app](#). A common pipeline workflow has the following steps:

1. A developer creates a pull request to make a change to the codebase.
2. Heroku automatically creates a [review app](#) for the pull request, allowing developers to test the change.
3. When the change is ready, it's merged into the codebase's test branch.
4. The test branch can [automatically be deployed](#) to the pipeline's staging app for further testing.
5. When the change is ready, a developer [promotes](#) the staging app to production, making it available to the app's end users.

In this lab, we're going to simulate the process mentioned above with Heroku learn the importance of seeding a database and how to populate Heroku's database for your Rails project, specifically using the "test_app" project that you have been working on in [previous tasks](#).

1. When going through this process, you can work with the GitHub repository that you created previously for [CI task](#).

2. (Start your docker and work in your terminal or powershell) Make sure you have dev, test, and **main** branches. We'll be using the test branch.

First, use `$ git status` to see if your git is in the correct branch, which should be 'test.'

If you haven't created your local branches, please do it now. For example, commands to create a test branch with git:

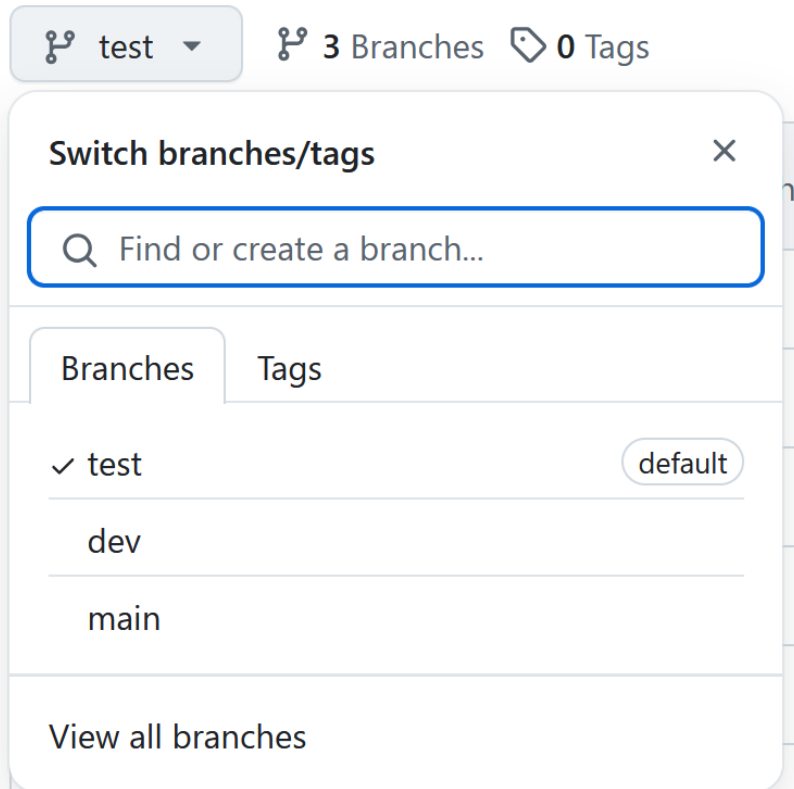
```
administrator@Administrators-MacBook-Pro-2:~/csc431/test/test_app$ git checkout -b test
Switched to a new branch 'test'
administrator@Administrators-MacBook-Pro-2:~/csc431/test/test_app$ git branch
  main
* test
administrator@Administrators-MacBook-Pro-2:~/csc431/test/test_app$ git push origin test
Total 0 (delta 0), reused 0 (delta 0)
remote:
remote: Create a pull request for 'test' on GitHub by visiting:
remote:   https://github.tamu.edu/dmartinez05/environment_test/pull/new/test
remote:
To github.tamu.edu:dmartinez05/environment_test.git
* [new branch]      test -> test
```

Note:

`$ git checkout -b <branch>` will create and switch to the new branch

`$ git checkout <branch>` will only switch to the branch

3. Create the three branches in your remote Github repo. Your Github should look like this:

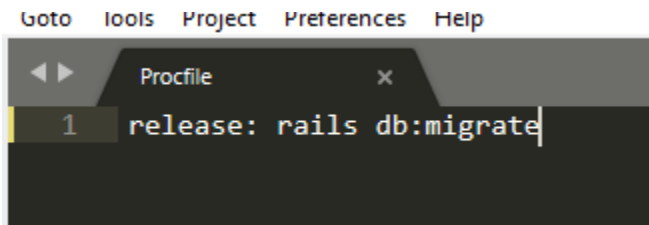


You don't need to create a new branch in your remote Github repository in advance because `$ git push origin <branch>` will do it for you if Github doesn't have a corresponding branch.

4. Create files necessary for deployment to Heroku.

Switch to your local test branch. Command - `$ git checkout test`

Create a file named Procfile in the root dir with this line of code

A screenshot of a code editor window. The title bar shows 'Goto', 'Tools', 'Project', 'Preferences', and 'Help'. The editor has a tab labeled 'Procfile' with a close button 'x'. The first line of the file is '1 release: rails db:migrate' with a cursor at the end of the line.

```
1 release: rails db:migrate
```

Create a file named app.json in the root dir with the sample code below. **Note: if heroku-24 (the most current) does not work, then try heroku-22**

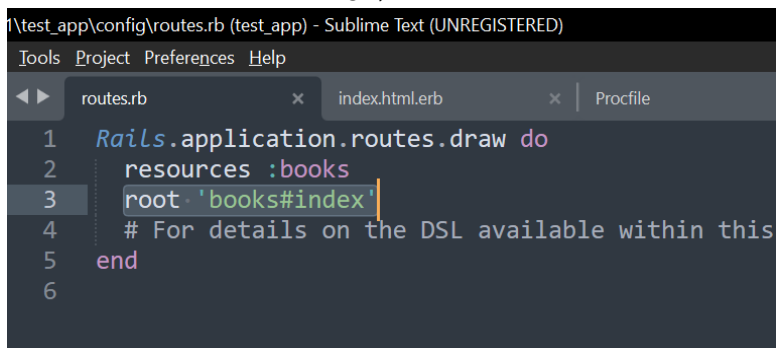
```
{
  "stack": "heroku-24",
  "addons": [
    "heroku-postgresql:in-dyno"
  ],
  "environments": {
    "review": {
      "addons": [
        "heroku-postgresql:in-dyno"
      ],
      "scripts": {
        "postdeploy": "bundle exec rails db:schema:load"
      },
      "env": {
        "RAILS_ENV": "review",
        "DISABLE_DATABASE_ENVIRONMENT_CHECK": "1"
      }
    },
    "buildpacks": [
      {
        "url": "heroku/nodejs"
      },
    ],
  },
}
```

```
    {
      "url": "heroku/ruby"
    }
  ],
  "env": {
    "NODE_ENV": "production",
    "YARN_PRODUCTION": "true"
  }
}
```

Create a file package.json in your root directory with the following code:

```
{
  "name": "book-collection-solution",
  "private": true,
  "dependencies": {},
  "engines": {
    "node": "20.x",
    "yarn": "1.22.x"
  },
  "version": "0.1.0"
}
```

5. Create a root route in config/routes.rb. (Follow all the instructions from here for both old and new docker image)



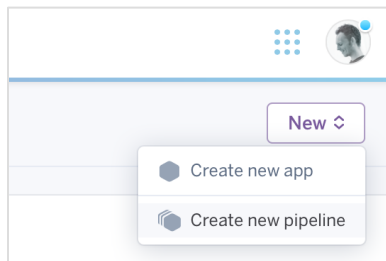
```
1 Rails.application.routes.draw do
2   resources :books
3   root 'books#index'
4   # For details on the DSL available within this
5   end
6
```

6. Save the changes and push it to origin/test

```
$ git add .  
$ git commit -m "descriptions of changes"  
$ git push origin test
```

7. From the Heroku Dashboard

Click the **New** button in the top right of your app list and select **Create new pipeline**:



Create New Pipeline

Introduction to pipelines

A pipeline is a group of Heroku apps that share the same codebase. Adding apps to a pipeline allows you to define stages and promote code between them. [Learn more](#).

Pipeline name **Required**

test-app

Pipeline owner

Yihao Xie (shayle@tamu.edu)

Connect to GitHub

Search for a repository to connect to

Yihao2020

enviro

Search

Missing a GitHub organization? [Ensure Heroku Dashboard has team access](#).

Yihao2020/environment_test

Connect

Create pipeline

Note: If there's no app in a pipeline, the pipeline will disappear. Therefore we need to configure some apps as default.

8. Create a Staging app

Using the Heroku dashboard, create a staging app as shown below by clicking on “Create new app...”:

STAGING

X Add app

→

Q Search for existing app...

Create new app...

Create New App

App name

Give this app a globally unique name. For example, acme-production-app.

stage-test-app-1-bhanu

✓

stage-test-app-1-bhanu is available

Location

Choose a Common Runtime region for this app. [Learn more.](#)

Common Runtime
CEDAR

United States

Common Runtime
CEDAR

Europe

Add this app to a pipeline

Pipelines form a continuous deployment workflow and enable additional features. All apps in the same pipeline must be the same generation. [Learn more.](#)

bookcollection3

Cedar

Choose a stage to add this app to

staging

Create a new application named **stage-test-app-1-bhanu** in the **bookcollection3** pipeline (**staging**) as a personal app.

Cancel

Create app

Click on “Create App” button above

9. Once you click Create app, then choose deployment method Github. Choose the desired repository, then hit connect.

Deployment method

 Heroku Git
Use Heroku CLI

 GitHub
Connect to GitHub

 Container Registry
Use Heroku CLI

Add Heroku Postgres as an Add-on

[Overview](#) [Resources](#) [Deploy](#) [Metrics](#) [Activity](#) [Access](#) [Settings](#)

Dynos

This app has no process types yet

Add a Procfile to your app in order to define its process types. [Learn more](#)

Add-ons

[Find more add-ons](#)

The add-on `heroku-postgresql` has been installed. Check out the documentation in its [Dev Center](#) article to get started.

 Heroku Postgres	Attached as DATABASE ↕	Provisioning	Essential 0	~\$0.007/hour ***
---	------------------------	--------------	-------------	-------------------

Estimated Monthly Cost

\$5.00

10. Add node.js and ruby as buildpacks, through Settings. Make sure the ruby add-on is last as shown below.

App Information

App Name

stage-test-app-1-bhanu

Region

United States

Stack

heroku-24

Frameworks

No frameworks detected

GitHub Repo

paulinewade/bookcollection3

Heroku Git URL

<https://git.heroku.com/stage-test-app-1-bhanu.git>

Generation

[object Object]

Buildpacks

Buildpacks are scripts that are run when your app is deployed. They are used to install dependencies for your app and configure your environment. [Find new buildpacks on Heroku Elements](#)

Your new buildpack configuration will be used when this app is next deployed.

Add buildpack

heroku/nodejs

X

heroku/ruby

X

11. Enable Review Apps. Do NOT select any options below for this assignment. Just click “Enable Review Apps” right away.

REVIEW APPS →

Enable Review Apps

Create apps for pull requests and branches on [Yihao2020/cicd-demo](#)

Enable Review Apps

Learn More

Ad

Configure Review Apps

- ☐ Create new review apps for new pull requests automatically
When enabled, every new pull request opened will create a Review app automatically and pull requests that are closed will be deleted.
- ☐ Wait for CI to pass
When enabled, new review apps will wait to deploy until Heroku CI passes.
- ☐ Destroy stale review apps automatically

Choose a region

All review apps will be created in this region

 United States

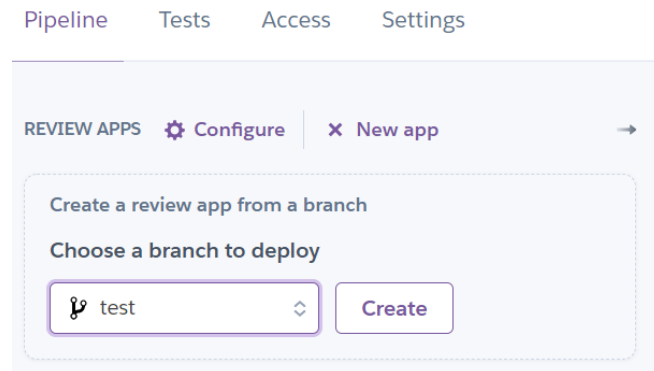


Review apps may incur dyno and add-on charges. These are invoiced to the app owner. [Learn More](#)

Cancel

Enable Review Apps

12. Click “New app” in Review Apps. Choose the test branch. After you click “Create”, Heroku will start deploying immediately.

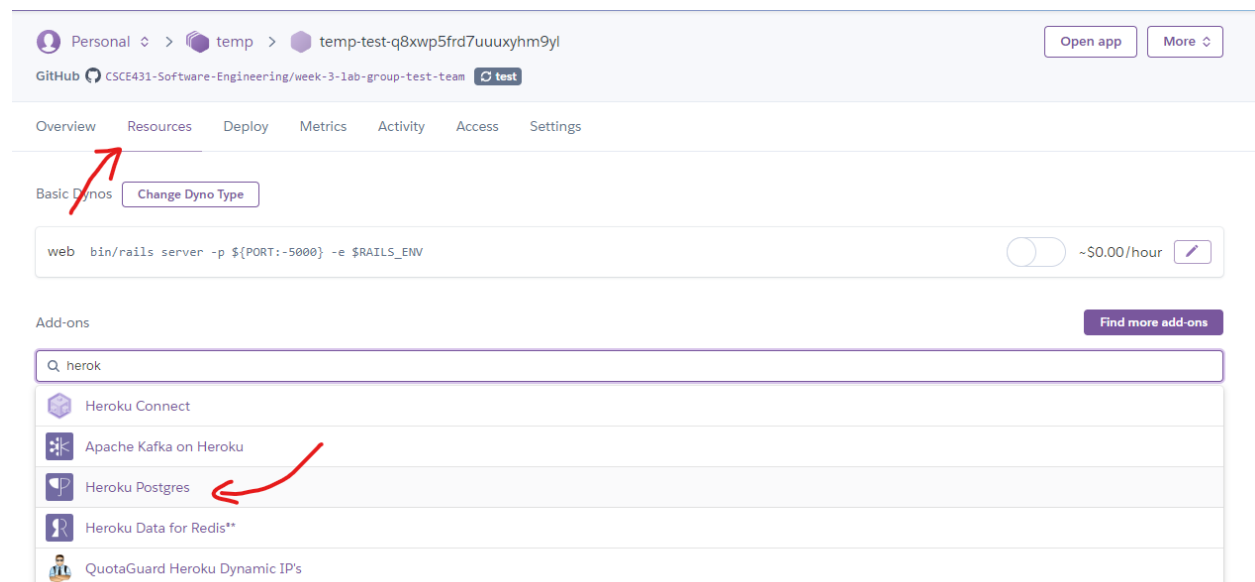


Note:

Your app.json and package.json should automatically provision the Heroku add-on and the node.js and ruby buildpack for your REVIEW area only.

If the release fails then check Resources to make sure Heroku Postgres is included in your add-ons, and Settings to make sure you have both the node.js and ruby buildpacks.

If not, you will have to add the Postgres database to the app in Resources. Click on the app and follow the below steps in the image. Add the node.js and ruby buildpacks as well in Settings.



Buildpacks

Buildpacks are scripts that are run when your app is deployed. They are used to install dependencies for your app and configure your environment. [Find new buildpacks on Heroku Elements](#)

			Add buildpack
≡	 heroku/nodejs		×
≡	 heroku/ruby		×

Re-deploy the app.

This should create a review app with its own app link that you can use to thoroughly test. You should see something like below.

REVIEW APPS  Configure | + New app 



bookcollecti-test-kjvo3qudszek

 test

0153a533

Deployed today at 4:08 PM

Open app 



13. Go back to your pipeline and click on 'Deploy a branch.'



stage-test-app-1-bhanu

Promote to production

Open app 



Configure automatic deploys...



Deploy a branch...

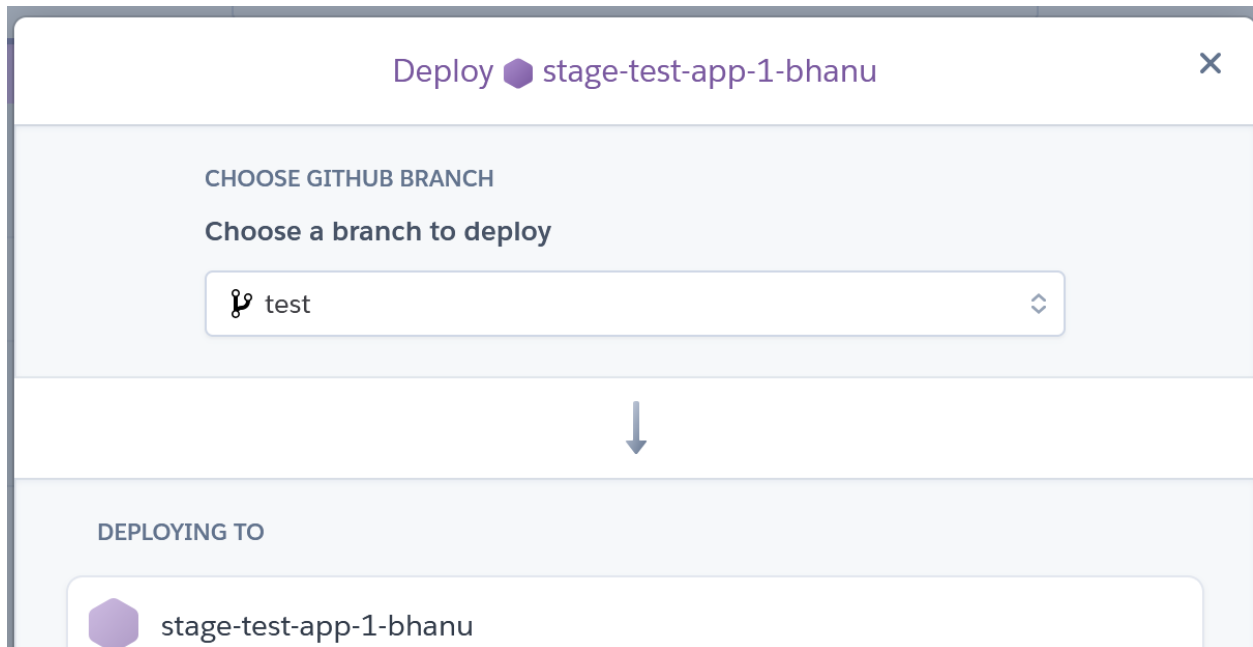
Move app to development

Move app to production

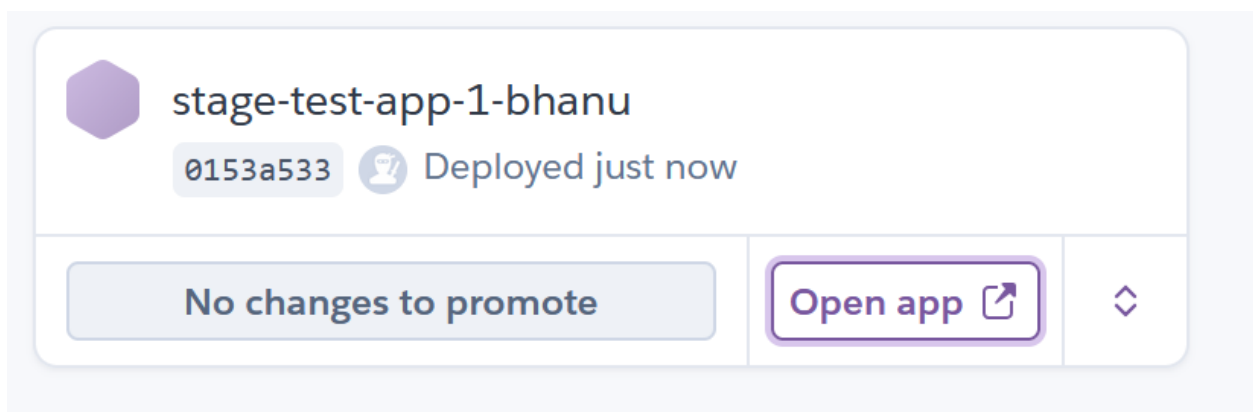


Remove from pipeline

14. Choose your test branch.



If everything works, then you should see below:



15. When the staging app is ready, open the app and test thoroughly. After you feel that your app is robust, then promote it to the production area by choosing “Move app to production.”

STAGING + Add app →

 stage-test-app-1-bhanu
0153a533  Deployed today at 5:11 PM

No changes to promote

Open app 



 Configure automatic deploys...

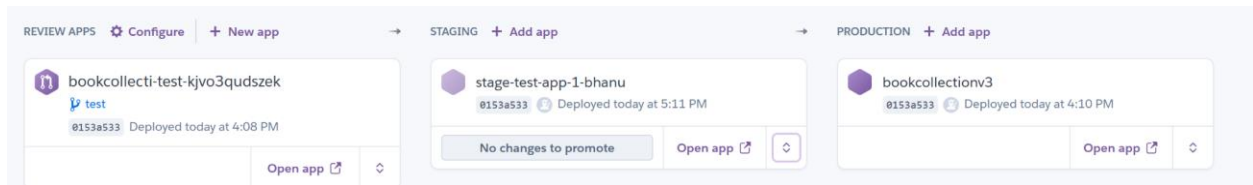
 Deploy a branch...

Move app to development

Move app to production

 Remove from pipeline

16. After your review, staging, and production apps are all deployed, your pipeline should look like below



The review, staging, and production apps will all have separate links. For example, the link to a staging app is shown below:

<https://stage-test-app-1-bhanu-569b7cdd983c.herokuapp.com/>

(See below for alternative instructions for creating a staging app using Heroku CLI, and for performing automatic deploy.)

Alternative approach of creating your staging app using Heroku CLI:

Install Heroku CLI from here based on your os.

(<https://devcenter.heroku.com/articles/heroku-cli>)

a. After installing type **heroku login** on your terminal.

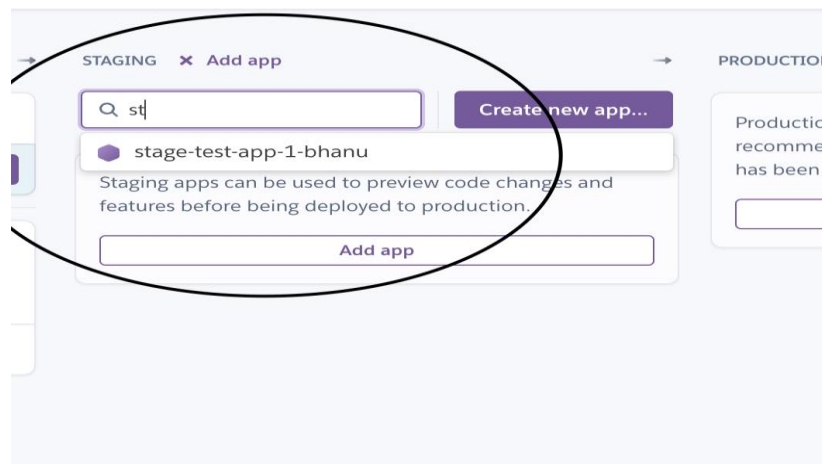
and follow instructions on the command line to log in.

b. Create an app for staging.

heroku create --stack heroku-24 stage-test-app-1-<name>

Example:: heroku create --stack heroku-24 stage-test-app-1-bhanu

Go to the Heroku dashboard, select add app under staging, and select this new app you just created with CLI.

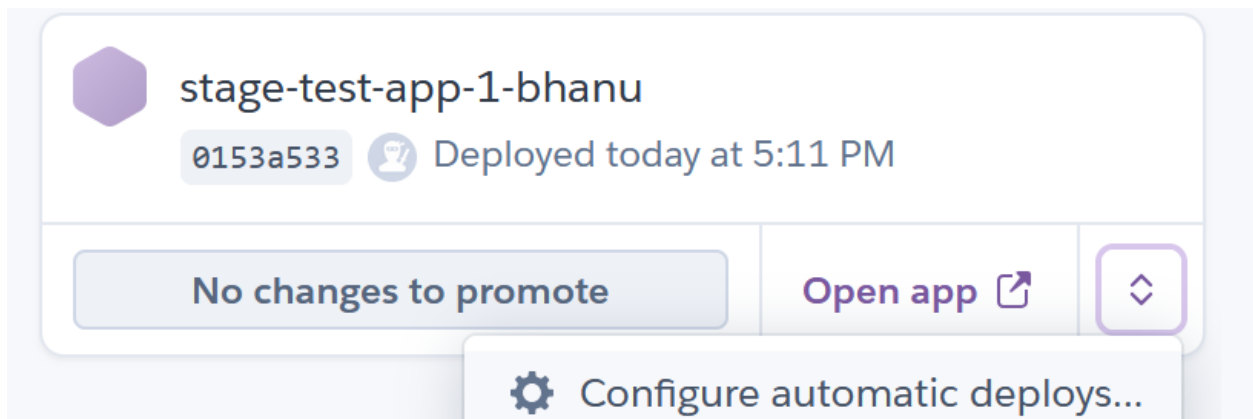


Click on "Create New app"

Follow the steps for the Heroku dashboard above starting with Step 11

Automatic deploy

Experiment adding automatic deploy to your staging area, using the instructions below



1. Let's make a small change to our app to see how the Heroku pipeline works.

You should remain in your local test branch (\$ git checkout test). You can be creative and change/add something that won't break your app. For example, add something into app/views/books/index.html.erb.

A screenshot of a code editor with a dark theme. The file 'index.html.erb' is open. The code contains HTML and ERB tags. Line 1: <p id="notice"><%= notice %></p>. Line 2: (empty). Line 3: <h1>Books</h1>. Line 4: <h1>This is a test version!</h1> (highlighted with a blue selection bar). Line 5: (empty). Line 6: <table>. Line 7: <thead>. Line 8: <tr>. Line 9: <th>Title</th>.

2. Then save the changes and push to the remote test branch.

You'll see that Heroku starts automatic deployment in the staging app. After the deployment is done, open the app and see if it works.

Database Seeding for Heroku

Objective

Database seeding is a crucial step in the development process as it allows you to pre-fill your database with initial data, making it easier to test and showcase your application's features. This lab will guide you through the steps of updating the Procfile, manually adding entries to the database, and creating a seed file. Additionally, you will explore the option of using the 'seed_dump' gem to automate the seeding process.

Steps

Step 1: Update Procfile

6. Modify the **Procfile** in your Rails project by adding the command **rails db:seed** to the existing commands.

```
1 release: rails db:migrate && rails db:seed
2
```

Note: use **RAILS_ENV=<ENV_NAME_HERE> rails db:seed** command to load the seed in the respective environment. (This will be used in the [submission](#))

Step 2: Locate the seed file

1. Locate the seed file at **db/seeds.rb** (the contents of the file are similar to the below image)

```
# This file should contain all the record creation needed to seed the database with its default values.
# The data can then be loaded with the bin/rails db:seed command (or created alongside the database with db:setup).
#
# Examples:
#
# movies = Movie.create([ { name: 'Star Wars' }, { name: 'Lord of the Rings' } ])
# Character.create(name: 'Luke', movie: movies.first)
```

Step 3: Manually Add Entries

1. Manually add entries of your choice (NOTE: you need to add at least 5 entries) to your database using your editor. Finally, the seed file should look something like this.

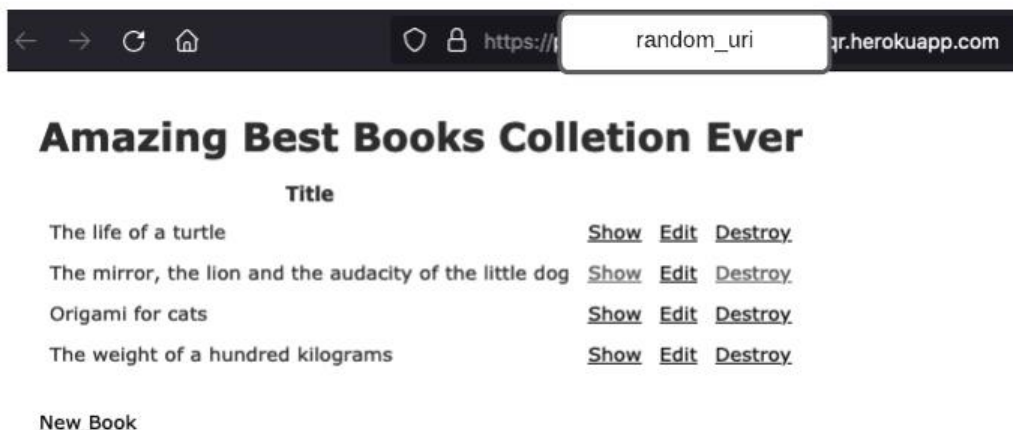
```
Book.create!([
  {title: "The life of a turtle"},
  {title: "The mirror, the lion and the audacity of the little dog"},
  {title: "Origami for cats"},
  {title: "The weight of a hundred kilograms"}
])
```

Step 4: Use 'seed_dump' Gem (Optional)

1. If desired, use the 'seed_dump' gem to automate the seeding process.
 - a. Add gem 'seed_dump' to your Gemfile.
 - b. Run `bundle install` to install the gem.
 - c. Automatically populate the seed file by running `rake db:seed:dump`

Step 5: Git Updates and Heroku

1. Add the modified files to Git (git add/commit/push).
2. Wait for Heroku to update with the new seed data (updated app shown below).



Step 6: Submission

Refer to [section 3 in the submission document](#) for submission details.

Additional Notes

1. Seeding is essential for creating a consistent environment for development, testing, and demonstration.
2. The 'seed_dump' gem automates the process of generating seed data from the existing database.

Conclusion

By following these steps, you will successfully seed your Heroku database, ensuring that your application has the necessary initial data for testing and demonstration purposes. This practice contributes to a smoother development workflow and a more robust application.